

Dokumentacja projektu OPUS-100 PL->EN

Adam Ropelewski 217518

Dawid Maliszewski 217276

Zakres projektu

Projekt realizuje zadanie tłumaczenia maszynowego z języka polskiego na język angielski z wykorzystaniem architektury Transformer typu encoder-decoder przy użyciu danych ze zbioru OPUS-100.

Najważniejszym założeniem projektu było samodzielne zaimplementowanie architektury Transformer NMT. Mechanizmy attention, warstwy enkodera i dekodera, maskowanie oraz generowanie tłumaczeń zostały zaimplementowane bezpośrednio w kodzie projektu bez używania gotowych rozwiązań z biblioteki PyTorch.

Główne pliki projektu:

- `configs/project_config.yaml` – konfiguracja modelu, treningu i ewaluacji,
- `src/model/transformer_nmt.py` – implementacja architektury,
- `scripts/train_model.py` – uruchomienie treningu,
- `scripts/evaluate_model.py` – ewaluacja i generowanie tłumaczeń.

Przygotowanie danych

Projekt wykorzystuje zbiór OPUS-100.

Przed treningiem dane zostały poddane czyszczeniu obejmującemu:

- usuwanie duplikatów,
- usuwanie identycznych par tłumaczeniowych,
- normalizację Unicode,
- usuwanie znaków sterujących,
- redukcję nadmiarowych białych znaków,
- odrzucanie przykładów o bardzo dużej różnicy długości.

Celem tego etapu było ograniczenie liczby błędnych lub zaszumionych przykładów trafiających do modelu.

Audyt danych z użyciem lokalnego LLM

Pomocniczo testowano lokalną Ollamę z modelem `qwen2.5:7b`. Model był wykorzystywany do ręcznego audytu wybranych par tłumaczeniowych oraz wyszukiwania podejrzanych przykładów.

Rozwiązanie to nie stanowiło części właściwego modelu tłumaczeniowego. Pełniło jedynie rolę narzędzia wspomagającego ocenę jakości danych.

Tokenizacja

Do tokenizacji wykorzystano bibliotekę SentencePiece.

Parametry tokenizera:

- typ: BPE,
- rozmiar słownika: 16000 tokenów,

- wspólny słownik dla języka angielskiego oraz języka polskiego,
- pełne pokrycie znaków.

Zastosowanie tokenizacji subword pozwala modelowi obsługiwać rzadkie słowa poprzez składanie ich z mniejszych fragmentów. Dzięki temu nie jest konieczne przechowywanie osobnego tokenu dla każdej możliwej formy fleksyjnej.

Analiza tokenizera wykazała:

- UNK rate = 0.0,
- średnio 12–14 tokenów subword na zdanie,
- 95% zdań mieści się poniżej 35 tokenów.

Batchowanie i maskowanie

Zdania mają różną długość, dlatego podczas tworzenia batchy są dopełniane tokenem <pad>.

Dla dekodera tworzona jest dodatkowo maska przyczynowa (causal mask), która uniemożliwia podgląd przyszłych tokenów podczas generowania tłumaczenia.

```
def make_causal_mask(size: int, device=None):
    return torch.triu(
        torch.ones((size, size), dtype=torch.bool, device=device),
        diagonal=1,
    )
```

Konfiguracja modelu

Najważniejsze parametry architektury zapisane są w klasie konfiguracyjnej:

```
@dataclass(frozen=True)
class TransformerNMTConfig:
    vocab_size: int
    pad_id: int = 0
    max_seq_len: int = 128
    d_model: int = 512
    nhead: int = 8
    num_encoder_layers: int = 6
    num_decoder_layers: int = 6
    dim_feedforward: int = 2048
    dropout: float = 0.1
    tie_decoder_embeddings: bool = True
```

Model wykorzystuje:

Parametr	Wartość
d_model	512
Liczba głów attention	8
Warstwy enkodera	6
Warstwy dekodera	6
Feed Forward	2048
Dropout	0.1

Architektura odpowiada klasycznemu wariantowi Transformer Base opisanemu przez Vaswaniego i współautorów.

Embeddingi i kodowanie pozycyjne

Transformer nie wykorzystuje rekurencji, dlatego pozycja tokenu musi zostać zakodowana jawnie.

Do embeddingów dodawane jest sinusoidalne kodowanie pozycyjne:

```
pe[:, 0::2] = torch.sin(position * div_term)
pe[:, 1::2] = torch.cos(position * div_term)
```

Pozwala to modelowi rozróżniać kolejność tokenów w zdaniu.

Multi-Head Attention

Podstawowym mechanizmem modelu jest Multi-Head Attention.

Dla każdego tokenu obliczane są trzy reprezentacje:

- Query (Q),
- Key (K),
- Value (V).

```
self.q = nn.Linear(d_model, d_model)
self.k = nn.Linear(d_model, d_model)
self.v = nn.Linear(d_model, d_model)
```

Następnie obliczane są wagi attention:

```
scores = torch.matmul(
    Q,
    K.transpose(-2, -1)
) / math.sqrt(self.head_dim)
```

```
attn = F.softmax(scores, dim=-1)
```

Mechanizm attention pozwala określić, które tokeny wejścia są najważniejsze dla aktualnie przetwarzanego elementu sekwencji.

Zastosowanie ośmiu głów attention umożliwia równoczesne modelowanie różnych zależności składniowych i semantycznych.

Feed Forward Network

Po warstwie attention każdy token przechodzi przez sieć Feed Forward.

```
class FeedForward(nn.Module):
    def forward(self, x):
        return self.linear2(
            self.dropout(
                F.relu(
                    self.linear1(x)
                )
            )
        )
```

Warstwa ta przetwarza reprezentację każdego tokenu niezależnie od pozostałych.

Enkoder

Każda warstwa enkodera składa się z:

- self-attention,
- Feed Forward Network,
- połączeń rezydualnych,
- LayerNorm.

```
x = self.norm1(
    x + self.dropout(
        self.attn(x, x, x, src_mask)
    )
)

x = self.norm2(
    x + self.dropout(
        self.ff(x)
    )
)
```

Enkoder buduje kontekstową reprezentację całego zdania źródłowego.

Dekoder

Warstwa dekodera zawiera trzy główne elementy:

1. Self-attention z maskowaniem przyszłych tokenów.
2. Cross-attention wykorzystujący reprezentację enkodera.
3. Feed Forward Network.

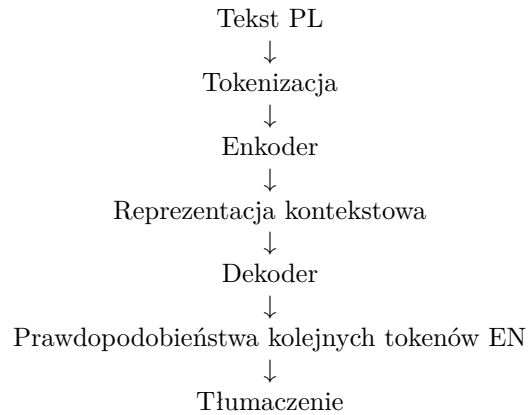
```
x = self.norm1(
    x + self.dropout(
        self.self_attn(x, x, x, tgt_mask)
    )
)

x = self.norm2(
    x + self.dropout(
        self.cross_attn(
            x,
            enc_out,
            enc_out,
            src_mask,
        )
    )
)
```

Cross-attention stanowi połączenie między zdaniem polskim a generowanym tłumaczeniem angielskim.

Przepływ danych

Przetwarzanie danych przebiega według schematu:



Wyjściem modelu są logity reprezentujące rozkład prawdopodobieństwa dla każdego tokenu słownika.

Wiązanie embeddingów

W modelu zastosowano współdzielenie wag embeddingu dekodera i warstwy wyjściowej.

```
if config.tie_decoder_embeddings:
    self.output_projection.weight = self.tgt_embedding.weight
```

Zmniejsza to liczbę parametrów oraz poprawia efektywność uczenia.

Trening

Najważniejsze parametry treningu:

```
precision: bf16
lr_peak: 1e-4
warmup_steps: 4000
weight_decay: 0.01
label_smoothing: 0.1
micro_batch_size: 48
grad_accum_steps: 8
```

Efektywny rozmiar batcha wynosi:

$$48 \times 8 = 384$$

Do funkcji kosztu wykorzystano Cross Entropy Loss z label smoothing.

```
nn.CrossEntropyLoss(
    label_smoothing=0.1,
    ignore_index=0,
)
```

Uczenie wykorzystuje harmonogram learning rate z fazą warmup oraz późniejszym spadkiem proporcjonalnym do odwrotności pierwiastka z numeru kroku.

Dodatkowo stosowane są:

- gradient clipping,
- checkpointy najlepszych modeli,
- early stopping.

Generowanie tłumaczeń

Podczas ewaluacji zastosowano Beam Search.

Parametry:

```
beam_size: 5
length_penalty: 1.0
max_new_tokens: 127
```

Beam Search utrzymuje jednocześnie kilka najbardziej prawdopodobnych hipotez zamiast wybierać pojedynczy token zachłannie.

```
top_scores, top_indices = torch.topk(
    candidate_scores,
    k=beam_size,
    dim=-1,
)
```

Pozwala to generować tłumaczenia o wyższej jakości niż standardowy greedy decoding.

Wyniki

Końcowe wyniki modelu:

Metryka	Wynik
SacreBLEU	20.35
chrF	41.82

Przykład poprawnego tłumaczenia:

PL: Każdy wie, że tu jesteś, Andre.
EN: Everyone knows you're here, Andre.

Przykład parafrazy zachowującej znaczenie:

PL: Byłeś dzisiaj wspaniały, Nate.
Reference: Hey, you were really great today, Nate.
Model: You were wonderful today, Nate.

Przykład błędu semantycznego:

PL: Mój stryj zagarnął moje ziemie i przysiągł wierność Duńczykom.
Reference: My uncle took my lands and pledged his allegiance to the Danes.
Model: My grandfather lost my land and popped the Duke.

Wnioski

Model poprawnie implementuje wszystkie podstawowe elementy architektury Transformer i dobrze radzi sobie z prostymi zdaniami oraz krótkimi wypowiedziami dialogowymi. Większe problemy pojawiają się przy długich zdaniach, rzadkich słowach, terminologii specjalistycznej oraz bardziej złożonych zależnościach składniowych.

Uzyskany wynik SacreBLEU wskazuje na działający model bazowy, jednak pozostaje jeszcze przestrzeń do poprawy jakości tłumaczeń.

Dalszy kierunek: Tatoeba

Po zakończeniu eksperymentów na OPUS-100 rozpoczęto pracę nad większym zbiorem Tatoeba.

Celem było sprawdzenie, czy ta sama architektura Transformer NMT osiągnie lepsze wyniki przy większej liczbie przykładów treningowych i dokładniejszym czyszczeniu danych.

Prace nad tym wariantem nadal trwają. Dotychczasowe wyniki sugerują, że dalsza poprawa jakości zależy głównie od jakości danych, długości treningu oraz skali modelu, a nie od zmian w samej architekturze.